

Day 15: Polymorphism

- Compile-time polymorphism
- Runtime polymorphism
- Method overloading vs overriding

Nuvvu ippativaraku nerchukunna topics:

- ☒ Classes
- ☒ Objects
- ☒ Methods
- ☒ Constructors
- ☒ Inheritance

Ippudu Java lo **most powerful concept** ki vacham.

Polymorphism

Word ni break cheddham.

Poly = Many (Chala)

Morph = Forms (Roopalu)

Polymorphism = One thing behaving in many forms.

Simple ga:

Oka method same name untundi.

Kani situation batti different ga work chestundi.

Example:

Think about **"Call"** button in phone.

- Friend ni call chesthe → Friend ki ring velthundi.
- Police number dial chesthe → Police connect avthundi.
- Ambulance dial chesthe → Ambulance connect avthundi.

Button okkate.

Behavior matram different.

Ide Polymorphism.

Real Life Example

Suppose oka person unnadu.

Office lo

He is

Employee.

Intlo

He is

Father.

Parents daggara

He is

Son.

Friends daggara

He is

Friend.

Person okkade.

Roles different.

Idi polymorphism.

Java lo Polymorphism Types

Java lo rendu types.

Polymorphism

|

|

| |

Compile Time Runtime

(Method (Method

Overloading) Overriding)

1. Compile-Time Polymorphism

Inkoka peru

Static Binding

or

Early Binding

Compiler compile chestappude decide chestundi.

Anduke compile-time polymorphism.

Idi

Method Overloading valla vastundi.

Method Overloading

Definition

Same class lo

Same method name

Different parameters.

Example

`add(int,int)`

`add(double,double)`

`add(int,int,int)`

Method name

`add()`

Same.

Parameters different.

Compiler parameter batti decide chestundi.

Example

```
class Calculator
{

    int add(int a,int b)
    {
        return a+b;
    }

    double add(double a,double b)
    {
        return a+b;
    }

    int add(int a,int b,int c)
    {
        return a+b+c;
    }

    public static void main(String args[])
    {

        Calculator c=new Calculator();

        System.out.println(c.add(10,20));

        System.out.println(c.add(5.5,6.5));

        System.out.println(c.add(1,2,3));
```

}

}

Output

30

12.0

6

Compiler ela decide chestundi?

Suppose

add(10,20)

Compiler searches

add(int,int)

Suppose

add(2.5,5.5)

Compiler searches

add(double,double)

Compile time lone decide aipoyindi.

Runtime varaku wait cheyyadu.

Anduke compile-time polymorphism.

Why Overloading?

Imagine overloading lekapothe.

add2()

add3()

addDouble()

addFloat()

addLong()

Ila hundreds methods create cheyyali.

Instead

add()

Enough.

Easy to remember.

Readable.

Professional.

Rules of Method Overloading

Rule 1

Method name same undali.

Correct

add()

add()

add()

Wrong

add()

sum()

Not overloading.

Rule 2

Parameters different undali.

Different means

Number change avvali

or

Datatype change avvali

or

Order change avvali.

Example 1

Number change

show(int)

show(int,int)

Correct.

Example 2

Datatype change

show(int)

show(double)

Correct.

Example 3

Order change

show(int,String)

show(String,int)

Correct.

Return type change chesthe overloading avthunda?

NO.

Wrong

`int add(int a,int b)`

`double add(int a,int b)`

Compiler Error.

Because parameters same.

Return type doesn't matter.

Why?

Suppose

`add(10,20)`

Compiler ki confusion.

Int call cheyyala?

Double call cheyyala?

Anduke error.

Constructor Overloading

Constructors kuda overload cheyochu.

`Student()`

`Student(int id)`

`Student(int id,String name)`

Idi kuda compile-time polymorphism.

Runtime Polymorphism

Second type.

Runtime lo decide chestundi.

Inkoka peru

Dynamic Binding

Late Binding

Idi

Method Overriding valla vastundi.

First understand Overriding

Already inheritance nerchukunnam.

Parent

↓

Child

Parent method ni child malli own implementation tho rayadam.

Example

```
class Animal
```

```
{
```

```
    void sound()
```

```
    {
```

```
        System.out.println("Animal makes sound");
```

```
    }
```

```
}
```

```
class Dog extends Animal
```

```
{
```

```
    @Override
```

```
void sound()
{
    System.out.println("Dog barks");
}

}
```

Now

Animal

↓

Dog

Parent

sound()

Child

sound()

Same method.

But implementation different.

Main Program

```
Animal a=new Dog();
```

```
a.sound();
```

Output

Dog barks

Question

Variable type

Animal

Why Dog method execute ayindi?

Answer

Runtime lo JVM object ni check chestundi.

Object

new Dog()

Kabatti

Dog method execute chestundi.

Compile time lo teliyadu.

Runtime lo decide ayindi.

Anduke Runtime Polymorphism.

Step by Step

Line

Animal a=new Dog();

Compiler sees

Animal

Compiler simply checks

"sound() Animal lo unda?"

Yes.

Compile success.

Execution start.

Runtime lo JVM checks

Object enti?

Dog

Kabatti

Dog version execute.

Memory Diagram

Animal a

|

|

Dog Object

Reference

Animal.

Actual Object

Dog.

Method

Dog di execute.

Why Runtime ani antaru?

Because

Compiler doesn't know final method.

Execution appude decide chestundi.

More Example

```
class Animal
```

```
{
```

```
    void sound()
```

```
    {
```

```
        System.out.println("Animal");
```

```
    }
```

```
}
```

```
class Dog extends Animal
{

    void sound()
    {
        System.out.println("Dog");
    }

}
```

```
class Cat extends Animal
{

    void sound()
    {
        System.out.println("Cat");
    }

}
```

Main

```
Animal a;
```

```
a=new Dog();
```

```
a.sound();
```

```
a=new Cat();
```

```
a.sound();
```

Output

Dog

Cat

Reference same.

Object changes.

Behavior changes.

This is real polymorphism.

Why Runtime Polymorphism is Powerful?

Imagine

Payment App.

Payment methods

- UPI
- Credit Card
- Debit Card
- Net Banking

All have

pay()

Parent

class Payment

{

void pay()

{

}

}

Children

UPI

Card

NetBanking

Each overrides

pay()

Now

Payment p;

Sometimes

new UPI()

Sometimes

new Card()

Sometimes

new NetBanking()

Always

p.pay();

Correct payment automatically execute avthundi.

Real projects lo ila use chestaru.

@Override Meaning

Nuvvu previous day adigina doubt ki answer.

@Override

Memory lo parent method ni preserve cheyyadam kaadu.

Adi **compiler ki oka instruction.**

Meaning:

"Ee method parent class method ni override cheyyadaniki rasanu. Check cheyyi."

Example

@Override

```
void sound()
```

```
{
```

```
}
```

Compiler checks

Parent lo

```
sound()
```

Unda?

Yes

Compile success.

Parent lo lekapothe

Error.

Without @Override

Program run avvochu.

Kani spelling mistake unte compiler detect cheyyakapovachu.

Example

Parent

```
void sound()
```

Child

```
void soud()
```

Spelling mistake.

Without

@Override

Compiler thinks

"New method create chesadu."

With

@Override

Compiler says

"No matching parent method."

Error.

So @Override safety kosam.

Method Overloading vs Overriding

Feature	Overloading	Overriding
Class	Same class	Parent & Child
Method Name	Same	Same
Parameters	Different	Same
Return Type	Can differ if parameters differ	Same or Covariant
Inheritance	Not required	Required
Decision	Compile Time	Runtime
Binding	Static	Dynamic

Easy Trick to Remember

Overloading

Think

One person

Many phone numbers.

Name same.

Numbers different.

Compiler chooses.

Overriding

Think

Father

Bike()

Child modifies bike.

Bike same.

Features changed.

Runtime decides whose bike.

Compile Time vs Runtime

Compile Time

Program Compile

↓

Method fixed

↓

Execution

Runtime

Compile

↓

Execution starts

↓

Object checked

↓

Method selected

Can We Override Static Methods?

No.

Static methods belong to the **class**, not to objects.

Runtime polymorphism works only with **objects**.

Example

class A

```
{  
    static void show()  
    {  
        System.out.println("A");  
    }  
}
```

class B extends A

```
{  
    static void show()  
    {  
        System.out.println("B");  
    }  
}
```

This is **method hiding**, not overriding.

Can Constructors Be Overridden?

No.

Reason:

Constructors are **not inherited**.

Only inherited methods can be overridden.

Can Private Methods Be Overridden?

No.

Private methods are visible only inside their own class.

Child class cannot access them.

So overriding is impossible.

Can Final Methods Be Overridden?

No.

```
final void display()
```

```
{
```

```
}
```

final means "This implementation is fixed."

Child cannot change it.

Can We Overload the main() Method?

Yes.

Example

```
public static void main(String[] args)
```

```
{
```

```
    main(10);
```

```
}
```

```
static void main(int x)
```

```
{
```

```
    System.out.println(x);
```

```
}
```

Output:

JVM always starts with:

```
public static void main(String[] args)
```

After that, your program can call the overloaded versions.

Interview Questions

1. What is Polymorphism?

One method/interface behaving in multiple forms depending on the situation.

2. Types of Polymorphism?

- Compile-time Polymorphism (Method Overloading)
 - Runtime Polymorphism (Method Overriding)
-

3. Which is faster?

Compile-time polymorphism is generally faster because the method is selected during compilation itself.

4. Which concept gives Runtime Polymorphism?

Method Overriding.

5. Which concept gives Compile-time Polymorphism?

Method Overloading.

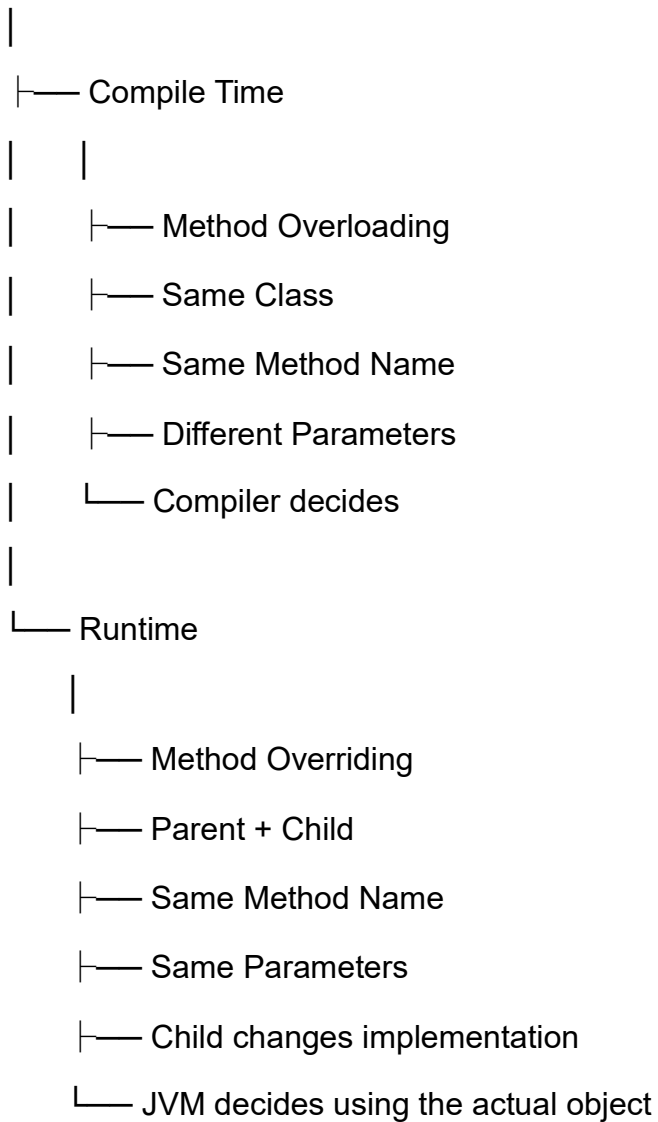
6. Is @Override mandatory?

No.

But it is highly recommended because it helps the compiler catch mistakes.

Memory Shortcut

POLYMORPHISM



Practice Programs (Must Do)

1. Create a Calculator class with overloaded add() methods for:
 - int + int
 - double + double
 - int + int + int
2. Create a Shape class with a draw() method. Create Circle, Rectangle, and Triangle classes that override draw(). Use:

Shape s;

s = new Circle();

s.draw();

```
s = new Rectangle();
```

```
s.draw();
```

```
s = new Triangle();
```

```
s.draw();
```

3. Create a Vehicle class with a start() method. Override it in Car and Bike, and observe how the correct method is called at runtime.